

EECS 498/CSE 598: Zero-Knowledge Proofs

Winter 2026

Lecture 14

Paul Grubbs

paulgrub@umich.edu

Beyster 4709

Agenda for this lecture

- Announcements
- What is a frontend?
- The “transcript” of a program
- Frontend programming models: ASIC vs. CPU
- ASIC-model frontends

Agenda for this lecture

- **Announcements**
- What is a frontend?
- The “transcript” of a program
- Frontend programming models: ASIC vs. CPU
- ASIC-model frontends

Announcements

- Project 3 online, due 3/20
- Theory track problem set 1 online, due 3/20
- 598 research track: email me

Agenda for this lecture

- Announcements
- **What is a frontend?**
- The “transcript” of a program
- Frontend programming models: ASIC vs. CPU
- ASIC-model frontends

What is a frontend?

First, use frontend component to:

1. Write human-readable program in ZK programming language.
 - captures requirements of problem in typical programming abstractions
 - Program P takes public input x and private witness w , outputs 0/1.
2. Run a compiler on P to transform it into an algebraic representation C .
 - Also need to generate (or have programmer write) an auxiliary program E that does “witness extension”: take x, w inputs for P , turn them into x', w' inputs for C . Need property that for all x, w , $P(x, w)=1 \Leftrightarrow C(x', w')=1$

The frontend's output is P , C , and E .

Agenda for this lecture

- Announcements
- What is a frontend?
- The “transcript” of a program
- Frontend programming models: ASIC vs. CPU
- ASIC-model frontends

Programs and transcripts

"transcript": values of internal variables

A simple example

```
def factorial(n):  
    acc = 1  
    for i=n -> 1:  
        acc = acc*i  
    return acc
```

A simple example

```
def factorial(n):  
    acc = 1  
    for i=n -> 1:  
        acc = acc*i  
    return acc
```

factorial(4):

4	acc	1	4	12	24	24
	i	4	3	2	1	1

The most important idea

We're verifying a transcript, not computing it!

Agenda for this lecture

- Announcements
- What is a frontend?
- The “transcript” of a program
- Frontend programming models: ASIC vs. CPU
- ASIC-model frontends

Important concepts of frontends

- Non-determinism: verify, don't compute
 - classic example: inverses
- Extending/enriching target model of computation
 - i.e., adding lookups, I-R1CS, custom gates
- Completeness and soundness of translation

Instead of computing a^{-1} , supply b
 $a \cdot b = 1$

• Formal methods

• PH tools

Programming models: ASIC vs. CPU

ASIC model: line-by-line translation of source program into R1CS

```
i=0;  
for (j=0; j<10; j++) {  
    i++;  
}
```

$\Rightarrow$

$$\left\{ \begin{array}{l} Z = 0, \\ Z_0 = Z + 1, \\ Z_1 = Z_0 + 1, \\ \vdots \\ Z_9 = Z_8 + 1 \end{array} \right\}$$

Note:
static single
assignment (SSA)
form

Programming models: ASIC vs. CPU

ASIC model: line-by-line translation of source program into R1CS

```
i=0;  
for (j=0; j<10; j++) {  
    i++;  
}
```

$\Rightarrow$

$$\left\{ \begin{array}{l} Z = 0, \\ Z_0 = Z + 1, \\ Z_1 = Z_0 + 1, \\ \vdots \\ Z_9 = Z_8 + 1 \end{array} \right\}$$

Note:
static single
assignment (SSA)
form

CPU model: compile program to “assembly” language

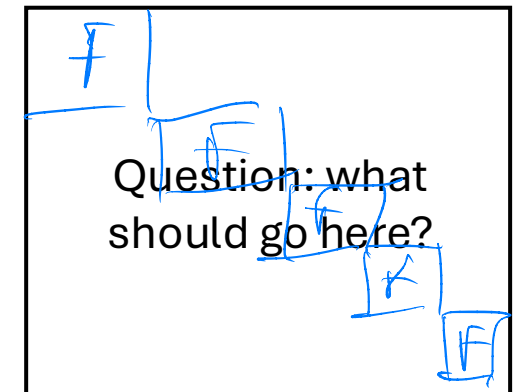
```
i=0;  
for (j=0; j<10; j++) {  
    i++;  
}
```



```
LDA #$01  
STA $0200  
LDA #$05  
STA $0201  
LDA #$08  
STA $0202
```



Unrolled "step"
fin + memory accesses



Agenda for this lecture

- Announcements
- What is a frontend?
- The “transcript” of a program
- Frontend programming models: ASIC vs. CPU
- **ASIC-model frontends**

ASIC-model frontends: the basics

What are some ASIC-model frontends?

Buffet, Pantry, Pequin, Pinocchio, xjSnark, CirC, Noir, Circom, Lurk, Bellman

ASIC-model frontends: the basics

What are some ASIC-model frontends?

Buffet, Pantry, Pequin, Pinocchio, xjSnark, CirC, Noir, Circom, Lurk, Bellman

Important distinction: embedded DSLs vs. compiled languages

ASIC-model frontends: the basics

What are some ASIC-model frontends?

Buffet, Pantry, Pequin, Pinocchio, xjSnark, CirC, Noir, Circom, Lurk, Bellman

Important distinction: embedded DSLs vs. compiled languages

By default, only have Field datatype for the backend field.

Need static bounds on all loops (unrolling)

Distinction between computations that are checked vs. unchecked

$\mathbb{F}_p$

$\Rightarrow$ $\hookrightarrow$ $\hookrightarrow$

(P4 is embedded DSL)